Data Mining Final Assignment

Eleonora Roncaglia, Bashier Gulzar, Elena Petridou

November 2025

1 Introduction

This report delves into our construction of a Locality-Sensitive Hashing (LSH) algorithm with the aim of finding users that have watched similar movies. An LSH algorithm increases the efficiency of similarity search by reducing the dimensionality of the data at hand by allowing us to narrow down "candidate pairs" of similar users before actually computing their similarity. In fact, MinHashing, which is its core functionality, allows us to have a much reduced representation of our data, as it decreases the number of dimensions to the chosen amount of hash functions (Leskovec et al., 2020, pp. 87–91).

In this report we will first explain the implementation of the algorithm itself and go over the data types used for each step, as data type choices are fundamental to optimised and efficient code. Subsequently, we will explain how we performed post-processing after candidate pairs of users have been identified. Finally, we will talk about parameter optimisation, and about our choices of signature length (number of hash functions) and bands.

2 Implementation and Data Structures

Almost all the functions in our module make use of `numpy` arrays. This choice stems from how the core code of **numpy** is developed. The core of the package consists of highly-optimised C code, which, being a low-level, compiled language, has significantly better performance than a high-level interpreter like Python. We also make use of sparse compressed row matrices for handling the original data, in order to speed up computation. We should also mention that our whole algorithm is implemented within a `class` called LSH, whose objects are instantiated with the attributes `self.data_, self.users_, self.movies_ and self.candidate_pairs_`. The most important of these are `self.data_`, which is our input data, and `self.candidate_pairs_`, which is initialised as an empty set.

2.1 Input data

The input data for the algorithm is first imported as a `numpy` array, and then converted to a compressed sparse row (CSR) matrix through the `scipy` package. The sparse matrix format is a memory-efficient way to store our user-movie matrix, considering that the original array is very sparse, due to the fact that zero entries -which would have been numerous- have been removed. In fact, we do not expect any users to have seen more than around 3000 movies, while the total number of movies is around 17000. This means that we could have at least 14000 0-values per row had we chosen to use the data in a conventional numpy matrix or pandas DataFrame, which would have inflated the size of the data-structure and thus slowed down the algorithm, as the omitted entries would have been stored as zeroes. CSR matrices fix this issue by saving a multi-dimensional array as three separate one-dimensional arrays: one containing the row positions of the ratings (each row represents a user), one containing the column positions of the ratings (each column represents a movie id), and one containing the non-zero ratings.

2.2 MinHashing

Hashing in the context of the MinHashing algorithm is a way for us to simulate what would happen if we were to randomly permute the movies (columns in our implementation) and then pick the first position in

our new permuted representation of our data which contained a non-zero rating. Each hash function $h_i(X)$, therefore corresponds to one "permutation" of the table. The CSR matrix contains no true zero values so in our implementation, we instead compute pseudo-permuted positions for all movies and for each hash function, and then select for each user, which movie out of the ones that they have watched was assigned the smallest pseudo-random position.

2.3 Hash functions

In order to compute the MinHash values for each user and store them, we first generate N_{funs} "hash functions". This N_{funs} is adjustable from the CLI, but we also discuss the optimal values in our implementation in section 5.

To make our pseudo-permutating hash functions, we use the Linear Congruential Generator (LCG) formula $(ax + b) \bmod c$, where a and b are random integers between 1 and the maximum of the values to be hashed (in our case, the total number of movies in the dataset), and c is a prime number slightly larger than said maximum. In particular, the choice of c is fundamental to the successful usage of the LCG generator. There are two essential conditions for c: it must be slightly bigger than the maximum value to hash, and it must be a prime number. The reason for the first condition is simply that we want the period c (or modulus) of the LCG generator to be bigger than the maximum value sought from the hash function, so that all numbers within the period can be generated. However, it must not be unnecessarily big so as to take up more memory than needed.

The second condition for our c is more tricky. An efficacious LCG generator is one that completes a full-period, or that generates $c - 1$ unique numbers, namely all the numbers within c, before repeating. To achieve such a generator, there are three conditions that need to be met (Hull & Dobell, 1962, p. 233):

1. a and c must be co-primes, meaning the only divisor they share must be 1
2. $a \equiv 1 \bmod p$, where p is a prime factor of c
3. $a \equiv 1 \bmod 4$ if 4 is a factor of c

The main reason for these conditions - in particular for the first one - is that if the modulus c is divisible by the multiplier a, the generator collapses (Pisharath & Math, 2003). The easiest and most computationally efficient solution to these conditions is to make c prime. A prime modulus entails that a and c are necessarily co-primes, as $a < c$ by design. For these reasons, we chose c to be the first prime number larger than the number of movies in our dataset.

Since the formula is the same for each hash function, we only need to store the parameters a, b, c in a `numpy` array of size $N_{\text{functions}} \times 3$, where each row stores a, b, c for one function.

2.4 Computing the MinHash table

We start by initialisimg a `numpy` array with size $N_{\text{users}} \times N_{\text{funs}}$. This will be the signature matrix (MinHash table) for each user that we will populate iteratively.

Subsequently, we use vectorised array operations to create a hash matrix (permuted table) of the movies. This is done by extracting our a, b, c parameters from the aforementioned 3-dimensional array, reshaping them, and computing the hash values with the formula $(ax + b) \bmod c$, where x are the movie ids. The resulting hash matrix has size $N_{\text{funs}} \times N_{\text{movies}}$, and each value represents the hash value for one movie obtained from one of the functions h_i, which conceptually would be its "position when randomly permuted $h_i(x)$.

The reason why we chose this implementation is that computing the hash matrix of all movies before computing the MinHash values for the users makes for much more efficient operations. Instead of calculating hash values for each user iteratively, we use the hash matrix as a 'lookup' table that holds all the pre-computed hash values for each movie.

Once this hash matrix is created, we can populate our signature (MinHash matrix) iteratively. For each user:

- Slice the hash matrix such that only the columns pertaining to movies the user has watched are considered

- Take the minimum hash value column-wise (across all hash functions), essentially creating a vector of MinHash values for all functions (one per function)
- Assign this vector to be the row of the signature matrix corresponding to this user

By doing this for all users, we essentially populate the $N_{\text{users}} \times N_{\text{funs}}$ MinHash (signature) matrix. Each row represents a user, and each column represents a hash function. Each value in the array is the minimum value of each hash function ("the first non-empty column position/first encountered movie watched by the user" if this was a permutation in the sparse context).

2.5 Banding

For banding, since the MinHash table has users as rows and hash values as columns, we need to band column-wise. To start with, we initialise an empty `numpy array` of size $N_{\text{users}} \times N_{\text{num bands}}$, (called `banded_data`) which we will populate iteratively.

Next, we define the column indices corresponding to each band. To do this, we divide the number of user-specified bands by the total number of columns in the signature matrix (the number of user-specified hash functions). Then, we loop through these indices and at each iteration subset the MinHash table per band, essentially constructing a local copy of it (which we will call a band for clarity), and is a `numpy` array with shape $N_{\text{users}} \times N_{\text{columns per band}}$. In order to speed up computation, we then hash this entire band using the `xxh32` function from the xxhash package, which computes hashing at RAM speeds. The whole hashed value for the band is then stored column-wise in the `banded_data` array.

2.6 Finding Candidate Pairs

To hold all candidate pairs we initialise an empty `set`, which we will update iteratively. It is important here to use a set, as updating it keeps no duplicate candidate pairs, and we know that duplicate candidate pairs are likely to occur in two different bands if two users are very similar.

Then, for each band, we use a **dictionary** to represent its buckets. The keys (hashed signature for this band) are retrieved iteratively as we loop through the rows, by selecting the subset of rows and columns corresponding to a user for a given band in the `banded_data` array. Then, we lookup the key in the dictionary which holds all our buckets. If the lookup is successful, we have a *collission*, which implies the existence of a possible candidate pair. We thus store this value in a list corresponding to this key in the dictionary, but *only if the list is not greater than our specified maximum bucket size.* (More on this in section 3.1).

Once we have partitioned all the hashed user signatures in a given band into our `buckets` dictionary, we can traverse through the bucket dictionary and create candidate pairs using the `combinations()` function from `itertools` to make tuples of length 2 for all unique combinations of pairs in each bucket. Using the set update operation allows us to ensure that during updating, no duplicate pairs will end up in the set.

2.7 Pseudo-Jaccard similarities

The pseudo-Jaccard similarities are stored in a `numpy` array with shape $N_{\text{pairs}} \times 3$ where each row contains the first user u_1, the second user u_2, and their similarity score between 0 and 1, with $u_1 < u_2$.

At first, we optimised our operations for speed by using vectorisation; but while this setup was very fast with 16GB of available RAM, it was too computationally intensive for a machine with only 8GB available.

As such, we decided to use banding here as well to preserve the speed of vectorisation while preventing the temporary objects stored in memory becoming unwieldy. As before, we slice the candidate pairs array into bands with length 10000 (meaning 10000 candidate pairs per band). Then, we iterate through the bands in order to compute the per-band per-row pseudo-Jaccard similarity using the vectorised `np.mean(s1 == s2, axis = 1)` operation, where $s1$, $s2$ are the users' signatures from our pre-computed Hash (user signature) matrix.

The pseudo-Jaccard similarity by relying on the following property:

$$J(u_1, u_2) \approx Pr\left(min\{\pi(S_{u_1})\} = \{min\{\pi(S_{u_2})\}\right) \tag{1}$$

which says that the probability that two users share a MinHash value (S_u) is approximately equal to their Jaccard similarity value (Broder et al., 1998). In our case, this translates to finding the percentage of common MinHash values between two users across all hash functions. In our algorithm, we extract the signatures for each user in a pair, and we compute the mean of the binary vector that represents equality between the signature values. Finally, we return a `numpy` array whose rows contain $u_1, u_2, J(u_1, u_2)$ for all bands.

3 Post-processing

In this section, we further explain some design choices taking place after candidate pairs have been created.

3.1 Choosing which buckets to test

As further discussed in section 5, the theoretically optimal parameters to find all pairs with a similarity higher than 0.5 are 28 bands with 5 columns each. This inevitably forces many users in the same bucket as it is quite likely that 5 users share the same 5 signatures in a band given a total of a 150 signatures. Computation of Jacard similarity for these buckets becomes increasingly expensive as $u(u-1)/2$, where u pairs have to be checked. To circumvent these expensive calculations a threshold to bucket size is applied: whenever bucket size grows larger than set threshold the bucket is skipped. The reasoning for this is that if two users are truly very similar, they will likely end up colliding across many buckets (given that there are 28 bands in our optimised approach). This means that we can more or less safely ignore buckets which exceeds a size of 50 users. This threshold can be adjusted via the CLI and does not impact the accuracy of our final results significantly, so we chose 50 as a trade-off between computing time and finding a high enough amount of candidate pairs.

3.2 Testing the validity of our discovered pairs

As we explained in the section 2.7, we do not compute the real Jaccard similarity score right away, as we rely on identity 1 to ensure that the pseudo-Jaccard similarity is approximately equal to the actual Jaccard similarity.

However, after narrowing down our candidate pairs using the pseudo-Jaccard value, we wanted to test whether these pairs have a true Jaccard score over the threshold. These are the pairs that end up being written to our results file.

Another interesting metric we have computed as part of this feature is the proportion of pairs from our narrowed-down pseudo-Jaccard pairs, that have a true Jaccard similarity score greater than 0.5.

3.3 Writing results to a text file

The last step of our LSH algorithm consists in writing the user pairs that exceed 0.5 Jaccard similarity. To do so, we slice the similarities array so as to keep only the user IDs of the pairs that exceed the similarity threshold, and we write the array to a text file, in the format u_1, u_2.

4 Results

For our submitted configuration (the values of which we stored in `config.py` for your convenience), we use seed 328429, 150 hash functions (signature length), 28 bands, and a maximum bucket size of 50. With these settings we find 183 truly similar pairs (with true Jaccard similarity score > 0.5), selected from among the 1683 pairs with pseudo-Jaccard similarity scores greater than 0.5 – giving us a true-positive proportion of 11%. The run takes about 60 seconds on a 2020 MacBook Air with an M1 chip with 8GB RAM.

5 Parameter trade-offs

In this section, we explain how and why we chose to optimise the parameters that we did. Our chief parameters of interest are: the signature length h – what we have up to now been referring to as the number of hash functions; the number of rows r - in our case columns - in each band of the signature matrix; and the number of bands b. Lastly, we also have the bucket threshold u, which limits how many users can be stored in a given bucket before it is deemed too large. In this section we briefly explain how different values of these parameters change the performance of our method.

5.1 Signature Length

As described in Leskovec et al. (2020), the number of signatures ultimately decides how precise we can be regarding finding candidate pairs. Signature length determines more or less how accurate our hashed approximation of the actual data will be. To illustrate this, consider a $1 \times N_{\text{movies}}$ vector representation of a user, u, where N_{movies} is some very large number (say 100,000). Now consider using the hash function $h(u) = (a \cdot u + b) \bmod c$, and, as described in sub-section 2.2, picking only the minimum value produced by this function, essentially "squishing" the representation of this user's movie preferences to just one scalar (a 1×1-sized representation). This representation does not closely approximate the original set of movies this user has watched at all.

However, if we were to repeat this process, say, 100 times, with and save each minimum value $h_1(u), ..., h_{100}(u)$ to a matrix, then we would have a 1×100 representation of this user's movie preferences. While not as large as the original u, this larger vector is a closer approximation than the 1×1 vector when using only one h. Since our approximation of the Jaccard similarity is computed using this MinHashing signature, therefore, the longer it is, the more likely our candidate pairs are to be truly similar.

Due to this reason, we optimised for the longest recommended signature length in the assignment specification: 150.

5.2 Number of bands and number of columns in each band

These signatures and therefore the precision of our initial guess are governed also by two related parameters: the amount of bands and amount of columns per band $h = c \cdot b$. A single MinHashed column matches between two users with the probability equal to their Jaccard similarity (Leskovec et al., 2020).

Let us imagine two users sharing a similarity s. Note that candidate pairs do not end up in the same bucket because of only a single matched column, but of an entire matched band of c columns. The probability that two users share an entire band is therefore: s^c – the probability that they share a column raised to the power of columns c in a band. Consequently the chance that two users do not share this particular band is: $1 - s^c$. The probability that two users do not share any band - the false negative rate - is thus: $(1 - s^c)^b$. Finally, the chance that two users with a similarity of s share at least one band and are therefore considered as a candidate pair is (Leskovec et al., 2020, pp. 101–102):

$$P(s) = 1 - (1 - s^c)^b.$$

The need for multiple bands is seen when we plug in $b = 1$, as this results into:

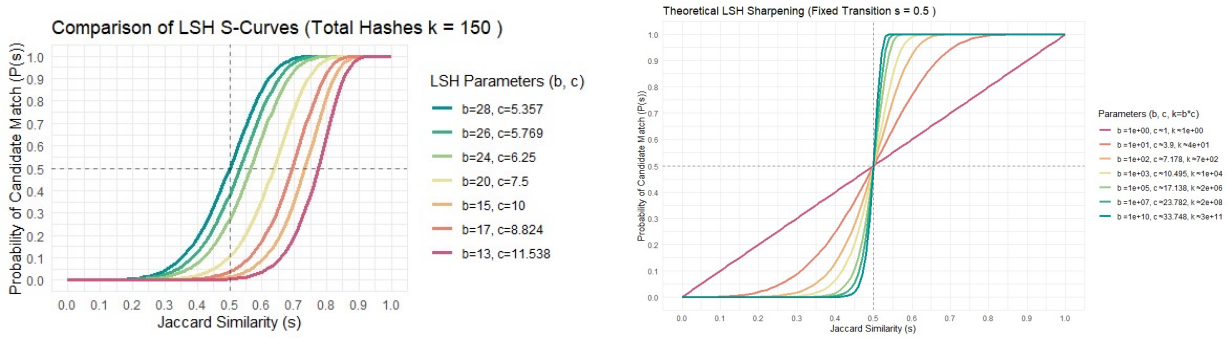
$$P(s) = 1 - (1 - s^c)^1 = s^c.$$

Effectively turning the chance that a certain pair is presented as a candidate pair into simply their respective Jaccard Similarity. Instead, we want to maximize the chance that pairs above a certain similarity s are presented as candidate pairs and pairs below this similarity threshold do not. I.e, we want to enforce $P(s_{threshold}) = 0.5$. In the case of this assignment we are asked to find pairs above 0.5 similarity, thus we want the decision boundary to be defined at $s_{threshold}$ of 0.5, turning our tuning issue into the form of:

$$0.5 = 1 - (1 - 0.5^c)^b.$$

Ideally, we would choose c and b such that $0.5 = 1 - (1 - 0.5^c)^b$ forms a perfect step function at the similarity threshold. The issue is that the amount of bands and columns per band necessarily multiplies to

the amount of signatures. To approach a perfect step function means approaching a near infinite amount of signatures 1b. To see this, one can plug in the the desired 0.5 probability point and simply rewrite $0.5 = 1 - (1 - 0.5^c)^b$ into $b = \frac{\ln(0.5)}{\ln(1-0.5^c)}$.



(a) Effect of number of bands on LSH curve

(b) Theoretical sharpening of LSH curve

Figure 1: Effect of parameter changes on LSH curves

Instead, choice of b and c must be constrained by the total amount of hash signatures f available. The assignment specified that a signature length in between 80 and 150 would suffice, but also specified that choice of parameters must be optimized to result in the same amount of candidate pairs over multiple random seeds. To this end, the maximum amount of 150 hash functions was chosen and parameters were optimized with respect to this. To calculate what c and b should be given $150 = c \cdot b$, we can solve $0.5 = 1 - (1 - 0.5^c)^b$, by rewriting $150 = c \cdot b$ into $c = \frac{150}{b}$ and solving for $b = \frac{\ln(0.5)}{\ln(1-0.5^{\frac{150}{b}})}$, yields approximately 28.0132. From this, c naturally follows to be $c = \frac{150}{28.0132} = 5.345614$

Choosing different bands while keeping signatures static essentially shifts the LSH curve (which is visualised in figure 1 to propose candidate pairs with either higher or lower similarities. While 5.345614 columns is theoretically correct, standard banding would not allow for float specification, as a band cannot contain a part of a column. There are multiple options to deal with this; our implementation adds the remaining columns to the last band. Concretely this means that our final choice of parameters lands on 28 bands and 5 columns per band, with the last band having 10 extra columns. Allowing for this changes the probability calculation for a candidate pair into

$$P(s) = 1 - \prod_{i=1}^{b_i} (1 - s^{c_i}).$$

The effect of this is negligible, as it only means that the last bucket has a slightly reduced chance of presenting candidate pairs as there are now more rows inset bucket.

References

Broder, A. Z., Charikar, M., Frieze, A. M., & Mitzenmacher, M. (1998). Min-wise independent permutations. *Proceedings of the thirtieth annual ACM symposium on Theory of computing*, 327–336.

Hull, T. E., & Dobell, A. R. (1962). Random number generators. *SIAM review*, *4*(3), 230–254.

Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). *Mining of massive data sets*. Cambridge university press.

Pisharath, J., & Math, N. (2003). Linear congruential number generators. *Newer Math, Fall*.